

CAPSTONE PROJECT COMPLETION PLAN REPORT

NTG-Agent vNext: Next-Generation Enterprise AI Agent Platform

Student Team	QUOTA
Students	Vu Gia An – s3926888 Nguyen Tien – s3924427 Tran Nguyen Quy Khang – s3929513 Nguyen Dinh Minh Chau – s3923010
Academic Supervisor	Tran Nhat Quang – RMIT Lecturer
Industry Supervisors	Nguyen Le Quynh Thien – Technical Manager
Company	NashTech Company

Table of Contents

Table of Contents.....	2
List of Figures.....	3
List of Tables.....	3
1. Revised Project Description and Scope.....	4
1.1. Scope Revision and Justification.....	4
2. Progress overview.....	6
2.1. Completed Work.....	6
The core assistant and its conversations.....	6
Specialist assistants working together (agents-as-tools).....	7
AG-UI[2] and A2UI[3].....	7
2.2. Outstanding Work Carried Forward.....	7
3. Deliverables and Schedule.....	7
3.1. Project Timeline.....	8
4. Success Criteria and Metrics.....	11
4.1. Evaluation Plan.....	11
5. Required Resources.....	13
6. Risk Management Plan.....	14
7. References.....	16
8. Appendices.....	17
A. Complete features.....	17
a. Core assistant.....	17
b. AI providers.....	18
c. TokenUsage / response-time records.....	19
d. Knowledge engine.....	20
e. Isolated stores. Two agents with different document libraries side by side.....	22
f. AG-UI/A2UI.....	24
g. Engineering discipline.....	25
B. GanttChart.....	26

List of Figures

Figure 1: Phase 2 — Advanced Agents & Product Completeness.....	8
Figure 2: Phase 3 — Evaluation & Production Hardening.....	9
Figure 3: Phase 4 — Polish, Evaluation Final & Showcase.....	9
Figure 4: user chat with the Agent.....	17
Figure 5: Agent generate a conversation title and reply to user.....	18
Figure 6: dedicated provider page for managing global providers that can be used by agents.....	18
Figure 7: Provider create/edit form that is compatible with multiple provider types and agents...	18
Figure 8: Choose provider and model for agent in Agent Setting.....	19
Figure 9: Track user token usage.....	19
Figure 10: Agent's response-time and its element.....	20
Figure 11: Upload a document for an agent.....	21
Figure 12: The UI indicates that the document is processed by LightRAG.....	21
Figure 13: Isolate knowledge of the Annie Agent and the Green Agent.....	22
Figure 14: Ask Annie agent about an information that it does not have in its knowledge while the Green agent has it.....	22
Figure 15: The Green agent can answer the question in detail thanks to the uploaded document.	23
Figure 16: The Annie agent still answer to the prompt but with the confession that its knowledge does not have the data.....	23
Figure 17: user prompt in the new webclient made specifically for AG-UI and A2UI.....	24
Figure 18: AG-UI feature puts information in a pre-design card.....	24
Figure 19: user asks agent to generate object from scratch and display it on chat box.....	25
Figure 20: Every PR will get to check with SonarCloud quality gate green.....	25
Figure 21: Full project timeline (Phases 2-4) with milestones and dependencies.....	26

List of Tables

Table 1: Scope revisions from Capstone A, with justification.....	4
Table 2: Completed work by platform area.....	6
Table 3: Project milestones, deliverables, and status.....	10
Table 4: Success criteria and measurable targets.....	11
Table 5: Evaluation plan and pass criteria per metric.....	12
Table 6: Required resources and current status.....	13
Table 7: Risk Management Table.....	14
Table 8: Risk register with priorities and mitigations.....	15

1. Revised Project Description and Scope

Our Capstone A proposal set out four headline pillars for the platform: an advanced document-search pipeline, a four-layer memory model, a multi-agent "planner and workers" system, and a streaming interface that could build parts of itself. Over this semester, testing each pillar against real company documents and the sponsor's priorities, we kept some as planned, delivered others in a simpler form, and deliberately retired the parts that were not earning their complexity. We also added three capabilities that were not in the original proposal.

1.1. Scope Revision and Justification

Table 1: Scope revisions from Capstone A, with justification

Pillar, as proposed	Where it stands now	Why it changed
Advanced document search — keyword-and-meaning search with an extra re-ranking step and inline citations	<i>Simplified.</i> We standardized on one graph-based engine (LightRAG[1]) using its own built-in search styles. The extra keyword-hybrid and re-ranking steps were not built, and citations are added where the model can. Importantly, a limitation flagged in the proposal, where every assistant shares a single knowledge store, is now solved by giving each assistant its own isolated store.	The graph engine on its own answered the sponsor's cross-document questions well, so the additional machinery would have added complexity for little measurable gain.
Four-layer memory model — including a long-term store that remembered each user across conversations, built on a separate search system	<i>Reduced.</i> Within-conversation memory, the recent messages plus a running summary, is delivered. The long-term per-user memory, and the separate search system behind it, were removed entirely.	For assistants that answer from documents, remembering individual users cost processing and infrastructure without a matching benefit, and removing it lets us retire a whole parallel search system, roughly halving what we maintain.

Pillar, as proposed	Where it stands now	Why it changed
Multi-agent teamwork[4] — a "planner" assistant that splits a request and hands the parts to specialist "worker" assistants over a formal message protocol	<i>Replaced.</i> A simpler "assistants-as-tools" model[4], in which a main assistant calls specialist assistants directly, like consulting an expert.	The simpler model met the same goal, combining specialists on complex requests, with far less coordination machinery to build, and it proved more stable in testing.
Self-composing interface — to be built natively inside our existing web framework	<i>Delivered differently.</i> Built on a dedicated new web client using an established interface toolkit that already speaks the same open standards.	No mature native option existed, so building one from scratch would have consumed time better spent on features; the dedicated client reached the same result sooner.

The first significant new capability is **assistant skills**[6]. A skill is a small, self-contained package of expertise. For example, how to handle an expense report or apply a particular company policy, written as a plain document with a name, a one-line summary, and step-by-step instructions. Administrators upload and manage these skills from a dedicated admin screen, and then choose which skills each assistant is allowed to use, all without changing any code. This capability is currently being built on a development branch covering the admin tools, per-assistant control, and both web interfaces — and is being finalised for release as part of Phase 2.

A second new capability is **fine-grained access control**. Beyond the basic distinction between administrators and ordinary users, an administrator can now grant each assistant to specific staff roles, so a member of staff only sees and can use the assistants their role is permitted to. The permission is enforced in two places: when a person chats with an assistant directly, and when one assistant calls another behind the scenes, so a restricted specialist cannot be reached indirectly by someone who is not allowed to use it. Like skills, this is built and tested on a development branch and is being finalised for release in Phase 2.

Two further additions round out the semester. A **shared cloud server** now hosts the knowledge engines; notably, cloud hosting was explicitly out of scope in the original proposal, but shared infrastructure became necessary for the team to run these tools at

scale. And a set of **practical improvements**, new assistants are set up in the background with their progress shown, administrators have role-based access, and the platform now supports a wider choice of five AI providers, including Anthropic.

2. Progress overview

Most of the revised platform described above is already built and working. This section walks through what is in place today, and then the two areas that are still in progress. Screenshot evidence for each completed area is collected in Appendix A (Figures 4-20).

2.1. Completed Work

Table 2: Completed work by platform area

The core assistant and its conversations	At the centre of the platform is a fully working chat assistant. Users hold flowing conversations that the system names and summarises automatically. The platform can connect to five different AI providers, including GitHub Models, OpenAI, Azure OpenAI, Google Gemini, and Anthropic, so an administrator can pick the model that best suits each assistant's purpose and budget. Importantly, every request records how much it used and how long it took, which gives us the raw measurements we rely on later to judge cost and speed. (Appendix A.a-A.c, Figures 4-10)
The knowledge engine	Each assistant can be given its own document library, and the graph-based engine answers questions by reasoning over those documents, including questions whose answer connects facts from several documents at once. This design also resolved a limitation we recorded in Capstone A, where every assistant had to share a single knowledge store: each assistant now runs its own isolated store, so their documents never mix. Administrators can add, export, or remove documents at any time, and the system shows the status of each document as it is prepared for searching. (Appendix A.d-A.e, Figures 11-16)
Running it reliably, and coordinating specialists	These knowledge tools now run on a shared cloud server, reached over a secure connection. Building on this, a user-facing assistant can now call on specialist assistants behind the scenes, and administrators manage those connections from an admin screen. Creating a new assistant no longer keeps the administrator waiting, either: the setup now happens in the background while its progress is shown on screen.

A richer interface, and engineering discipline	A new web interface presents rich, interactive results instead of plain text, and can let the AI compose parts of the screen itself; it shares a single sign-in with the existing interface, so the experience is seamless. Access is role-based, so administrators and ordinary users see different capabilities. Underpinning all of this, we have kept a steady engineering discipline throughout — more than 280 automated tests, automatic quality and security checks on every change, and a dedicated round of security hardening. (Appendix A.f-A.g, Figures 17-20)
Specialist assistants working together (agents-as-tools)	This feature is now in place. The main assistant treats each specialist as a tool it can invoke, then combines their answers into a single reply, and administrators decide from an admin screen which specialists each assistant is allowed to call. This lets the platform break a difficult, multi-part request into focused pieces, without the user ever seeing the hand-off.
AG-UI[2] and A2UI[3]	It is built on two emerging open standards: AG-UI[2], which streams an assistant's progress and results to the screen as they happen, and A2UI[3], which lets an assistant describe interactive on-screen elements that the interface then draws. Together these let the interface show live activity and structured, interactive results, and even let the assistant compose parts of the screen itself, rather than replying with a wall of text. It shares a single sign-in with the existing interface, and access is role-based, so administrators and ordinary users see different capabilities. (Appendix A.f, Figures 17-19)

2.2. Outstanding Work Carried Forward

Three areas are still in progress, and all are scheduled into the plan below. The first is finishing two capabilities that are already built and working on development branches: **assistant skills** and **fine-grained access control**. The remaining work is to finalise them and merge it into the main product during Phase 2. The second is a **formal evaluation of answer quality**: the measurement setup is ready, but the evaluation itself has not yet been run. After we retired the older search system, we reshaped this evaluation into a comparison of the graph engine's four different answering styles, to find which performs best on the sponsor's documents. The third is **preparing the platform for real-world deployment**.

3. Deliverables and Schedule

Since Capstone A, we have re-scoped the remaining schedule to match the revised feature set described in Section 1. Phase 2 (20/07 - 26/08) completes the product: the

assistant skills and fine-grained access control branches are finalised and merged to main, time-to-first-token (TTFT) telemetry is added to the streaming path, admins gain multi-provider model fetching and enter a third-party provider endpoint (OpenAI, Azure OpenAI, Gemini, DeepSeek...), the system automatically fetches the available models and the knowledge layer moves to shared knowledge spaces, where one LightRAG[1] container can serve multiple agents and one agent can draw on multiple containers. The phase closes at the Feature Complete milestone (26/08). Phase 3 (17/08 - 09/09) hardens the platform for real-world operation and establishes the evaluation baseline: deployment runbook, monitoring and health checks, backup and restore, and the retrieval benchmark comparing LightRAG's[1] four query modes on the fixed corpus, with performance metrics (TTFT, ingestion latency, cost per conversation) collected from telemetry the platform already records. It ends at the Production Ready milestone (09/09). Phase 4 (07/09 - 22/09) is polish and delivery: bug fixing, final documentation and sponsor handover, the final report and poster, an uptime dry run one week before the Showcase, and rehearsal, closing at the Showcase Delivery milestone (22/09). The complete timeline across all phases, with milestones and dependencies, is shown in Appendix B (Figure 21).

3.1. Project Timeline



Figure 1: Phase 2 — Advanced Agents & Product Completeness

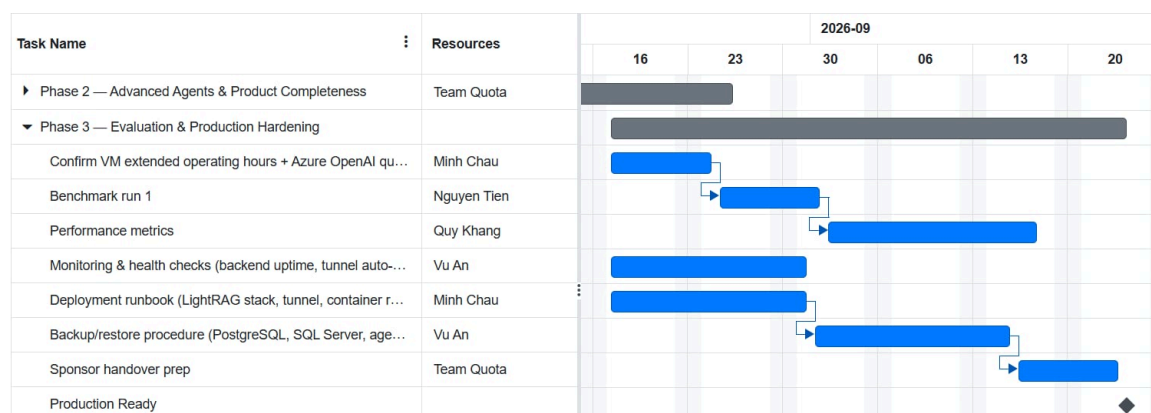


Figure 2: Phase 3 — Evaluation & Production Hardening

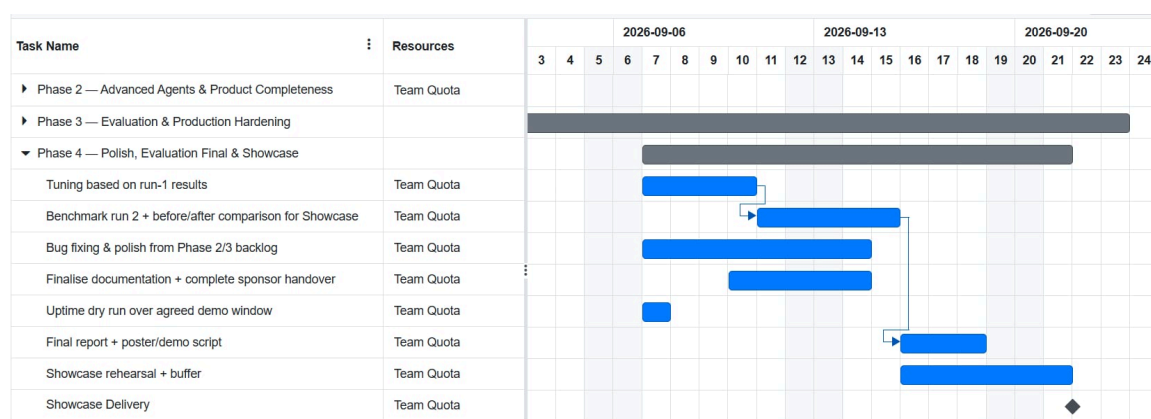


Figure 3: Phase 4 — Polish, Evaluation Final & Showcase

Table 3: Project milestones, deliverables, and status

Milestone	Features / functions delivered	Target date	Status
Phase 2 — Advanced agents & feature completeness	Assistant skills release, fine-grained access control release, multi-provider model fetch, shared knowledge spaces (many-to-many agent-container mapping), AG-UI[2]/A2UI[3] parity with Blazor client, knowledge admin UX (status, re-index, delete), anonymous-user limits polish, TTFT telemetry	26/08/2026	On-going
Phase 3 — Evaluation & production hardening	Deployment runbook, monitoring & health checks, backup/restore, VM hours & quota confirmation, retrieval benchmark run (4 LightRAG[1] query modes), performance metrics, sponsor handover prep	09/09/2026	On-hold
Phase 4 — Polish, evaluation final & Showcase	Bug fixing & polish, final documentation & sponsor handover, final report, poster/demo script, uptime dry run, Showcase rehearsal	22/09/2026	On-hold

4. Success Criteria and Metrics

Table 4: Success criteria and measurable targets

Metric	Aligned feature	How measured (observable)	Target
Retrieval relevance (e.g. hit-rate@k / human-rated correctness on a fixed Q&A set)[5]	M2 LightRAG[1] query-mode benchmark	Fixed benchmark corpus + query set, blind scoring by 2 raters	Hybrid ≥ 0.80 hit-rate@5 and mean rating $\geq 4.0/5$; hybrid $\geq$ best single mode
Ingestion latency per document (P50/P95 by size band)	Knowledge pipeline (M1)	Timestamps already tracked by ingestion status records	P95 ≤ 20 s (<100 KB), ≤ 90 s (100 KB–1 MB), ≤ 5 min (>1 MB)
Chat time-to-first-token and full-response time	Orchestrator streaming	Token-usage/response-time telemetry already persisted per request	TTFT P50 ≤ 1.5 s, P95 ≤ 4 s; full response P50 ≤ 12 s (default model)
Availability of knowledge backend during demo window	Remote VM + tunnel (M1)	Health checks / uptime log over an agreed window	100% within the agreed window; any tunnel drop auto-recovers within 60 s
Cost per conversation (tokens $\times$ provider price)	Multi-provider agents	TokenUsage table aggregation	Baseline established M1; median $\leq \$0.03$ for a 5-turn conversation on the default provider
Code quality gates green	Whole codebase	SonarCloud quality gate + CodeQL on main	pass on every merge

4.1. Evaluation Plan

Table 5: Evaluation plan and pass criteria per metric

Retrieval relevance	With two members as independent raters, run the fixed benchmark corpus and query set through each LightRAG[1] query mode (naive / local / global / hybrid); record hit-rate@5 automatically, and have both raters blind-score answer correctness on a 1–5 scale without seeing which mode produced each answer. Average the two ratings and resolve any gap of ≥ 2 points by discussion. When: at the retrieval-evaluation milestone (M2), as a single benchmark run across all four modes; the results are reported at the Showcase. Pass: hybrid ≥ 0.80 hit-rate@5, mean rating $\geq 4.0/5$, and hybrid $\geq$ the best single mode.
Ingestion latency per document	We will read the created-to-ready timestamps the ingestion-status records already capture; sort documents into the three size bands and compute P50/P95 per band over a representative sample when the corpus is loaded for the benchmark (M1 $\rightarrow$ M2). Pass: P95 ≤ 20 s (< 100 KB), ≤ 90 s (100 KB–1 MB), ≤ 5 min (> 1 MB).
Chat response speed (time-to-first-token and full response)	Full-response time is already recorded for every request and aggregated from the response-time telemetry; first-token timing is added to the streaming path as a small Phase 2 instrumentation step, then read from the same telemetry. Sample across the default model under normal load and compute P50/P95. Pass: TTFT P50 ≤ 1.5 s and P95 ≤ 4 s; full response P50 ≤ 12s.
Availability of the knowledge backend during the demo window	We agree on a demo window with the sponsor; a scheduled health-check samples the tunnel and backend at a fixed interval across that window and logs uptime, including the recovery time after any drop. We will perform a dry run one week before, then live across the Showcase window. Pass: reachable 100% of the agreed window, with any tunnel drop auto-recovering within 60 s.
Cost per conversation	We will aggregate the recorded token counts $\times$ the provider's

	published price over a set of representative five-turn conversations on the default provider. The baseline is established at M1 and re-check before the Showcase. Pass: median $\leq$ US\$0.03 per five-turn conversation.
Code quality gates green	The SonarCloud quality gate and CodeQL run automatically on every pull request to main, and a failing gate blocks the merge. When: continuous. Pass: green on every merge.

5. Required Resources

Table 6: Required resources and current status

Resource	Purpose (task it enables)	Status / ask
Azure Ubuntu VM (Docker host for LightRAG[1] stack)	Per-agent knowledge containers + PostgreSQL (M1, M2)	Provisioned; operating window currently limited (Mon business hours; other days 17:00–23:59 ICT) — extended hours needed around evaluation and Showcase
Azure OpenAI quota (chat + embeddings deployments)	Agent responses and LightRAG[1] ingestion embeddings (all milestones)	Active; monitor rate limits during benchmark runs
GitHub Models / other provider API keys	Multi-provider agent support and default agent	Active; tokens expire — rotation owner needed
SQL Server (containerised, dev)	Application data	In place via Aspire
Team time: 4 members $\times$ 20h/week	All milestones	Confirmed.

6. Risk Management Plan

The below table serves as a guide to identify the severity, likelihood, and level of each risk adapted from MIL-STD-882E.

Table 7: Risk Management Table

Severity (S)	Likelihood (L)				
	Improbable (1)	Remote (2)	Occasionally (3)	Probable (4)	Frequent (5)
Catastrophic (5)	5	10	15	20	25
Critical (4)	4	8	12	16	20
Significant (3)	3	6	9	12	15
Marginal (2)	2	4	6	8	10
Negligible(1)	1	2	3	4	5

The multiplicity of severity and likelihood factor produce the following level of classification:

- **>= 15 (EXTREME - E):** Immediate response required from every team member, regardless of their availability.
- **10 - 12 (HIGH - H):** Weekly monitoring by multiple team members is required. The problem and appropriate solutions will be the top priority in upcoming online and offline meetings.
- **4 - 9 (MEDIUM - M):** Monitoring is required only by the entrusted team member(s), and progress will be reported or discussed using the chosen work management platform (e.g., Microsoft Teams).
- **1 - 3 (LOW - L):** Monitoring is required only by the entrusted team member. The situation's progress can be updated or mentioned once higher-priority risks have been mitigated

Table 8: Risk register with priorities and mitigations

#	Risk (component / task)	Example already observed	S x L => Level	Mitigation / workaround
1	Knowledge VM unavailable outside operating hours (remote LightRAG[1], M1/M2)	Uploads failed while the VM was deallocated during development	5*4 = 20 => E	Agree extended hours for evaluation/Showcase; keep the local docker-compose fallback runnable; health-check script (./ntg lightrag-tunnel status) before demos
2	Credential/configuration drift (API keys, container-baked passwords)	Expired GitHub Models token caused chat 401; stale Postgres password left agent containers crash-looping	4*3 = 12 => H	Single owner for secret rotation; startup validation that fails fast with clear errors (LightRAG[1] startup now tolerates an absent Docker daemon/tunnel); documented recovery runbook (recreate containers)
3	LLM provider rate limits/cost spikes during benchmark (M2)	GitHub Models 8k-token/RPM caps previously caused search timeouts	3*4 = 12 => H	Use Azure OpenAI deployments with agreed quota for benchmarks; batch and cache embedding calls; monitor TokenUsage dashboards
4	Environment parity failures (SQL Server volume format, WSL clock drift)	2025-format data volume crashed 2022 containers; negative timespan from clock skew failed chat requests	3*3 = 9 => M	Pin image tags per volume; monotonic-clock timing (done); document per-OS setup in the repo
5	Late-merge integration breakage between parallel feature branches	feature/memory-4 vs AG-UI[2] merge produced runtime-only breakage (dual DbContext migration failure); since resolved by merging main weekly	3*2 = 6 => M	Merge main weekly into feature branches; CI runs migrations; PR review checklist

#	Risk (component / task)	Example already observed	S x L => Level	Mitigation / workaround
6	Team availability around exam periods (all milestones)	Overlapping exams and the deadline at RMIT.	2*1 = 2 => L	Front-load M1/M2; buffer week before Showcase; owner + backup per milestone
7	Inconsistent developing environment	Both appsettings file and secrets introduce overlapping environment variables without a unified .env file, creating drift among team members leading to inconsistent testing results.	3*1 = 3 => L	Manually compare edit appsettings file and secret between team members. Considering self-hosting a unified vault as secret managers.

7. References

[1] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, "LightRAG: Simple and fast retrieval-augmented generation," in Findings of the Assoc. for Computational Linguistics: EMNLP 2025, Suzhou, China, Nov. 2025, pp. 10746–10761.

[2] "AG-UI: The Agent-User Interaction Protocol," AG-UI Protocol. Accessed: Jun. 20, 2026. [Online]. Available: <https://docs.ag-ui.com/introduction>

[3] "Introducing A2UI: An open project for agent-driven interfaces," Google Developers Blog, Dec. 15, 2025. Accessed: Jul. 20, 2026. [Online]. Available: <https://developers.googleblog.com/introducing-a2ui-an-open-project-for-agent-driven-interfaces/>

[4] Anthropic, "Building effective agents," Anthropic, Dec. 2024. Accessed: Jul. 20, 2026. [Online]. Available: <https://www.anthropic.com/engineering/building-effective-agents>

[5] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," arXiv:2312.10997, 2023.

[6] Microsoft, "Skills," Microsoft Learn: Agent Framework documentation. Accessed: Jul. 20, 2026. [Online]. Available:

<https://learn.microsoft.com/en-us/agent-framework/agents/skills>

8. Appendices

A. Complete features

a. Core assistant

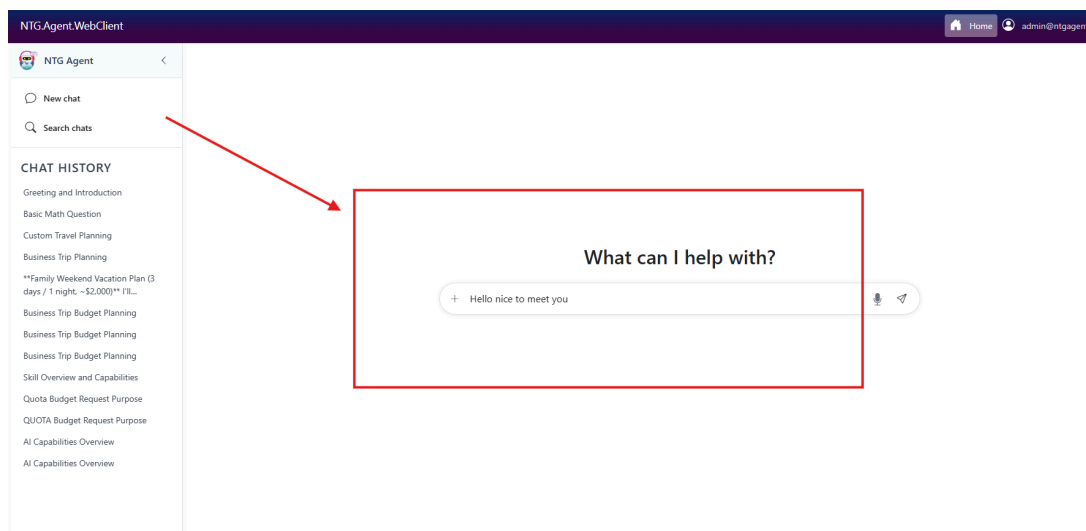


Figure 4: user chat with the Agent

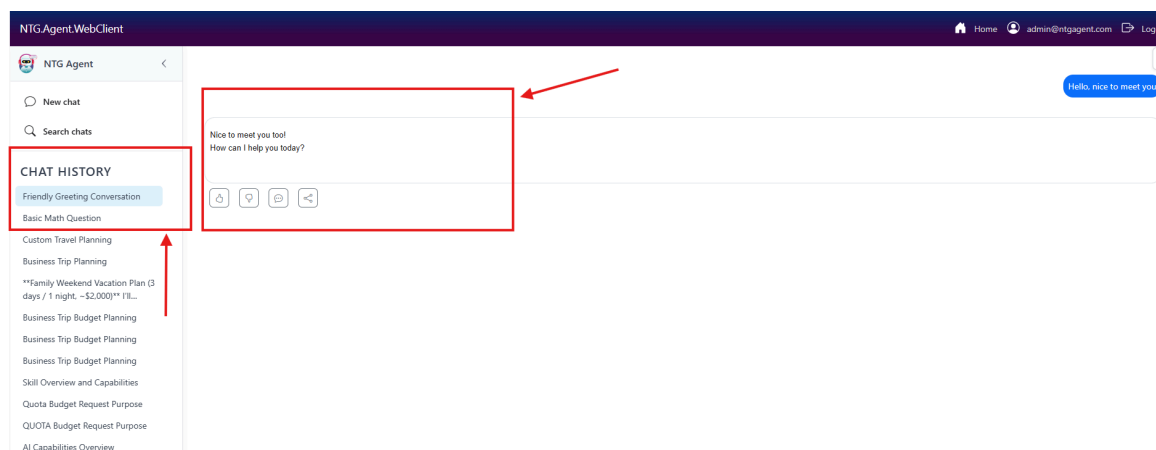


Figure 5: Agent generate a conversation title and reply to user

b. AI providers

NTG.Agent.Admin

Agent Management

Providers

Users Management

Tags Management

Token Usage

admin@ntgagent.com

Logout

Providers

Add Provider

Name	Type	Endpoint	API Key	Default Model	Agents	Actions
Default Provider	OpenAI	—	—	gpt-4o	0	
Azure	Azure OpenAI	https://mlit-capstone-2026-hcm--resource.openai.azure.com/openai/v1	FK7gEm***GuSx	gpt-5.1	3	

Figure 6: dedicated provider page for managing global providers that can be used by agents.

Edit Provider

Name *

Deepseek

Provider Type *

OpenAI Compatible

Endpoint

https://api.deepseek.com/v1

API Key

.....

Default Model

deepseek-v4-pro

✓ Connected successfully. 2 models found.
Models: deepseek-v4-flash, deepseek-v4-pro

Test Connection

Cancel

Save

Figure 7: Provider create/edit form that is compatible with multiple provider types and agents

⚙️ Agent Settings
✂️ Tools 0/0
📖 Knowledge Base 0

Agent Configuration

Configure the behavior and personality of your agent

Provider (Optional)
 Deepseek (OpenAI Compatible)

Model (Optional)
 Use default (deepseek-v4-pro)

Description

A brief description of the agent's purpose to help identify it.

Enter a brief description of the agent...

Figure 8: Choose provider and model for agent in Agent Setting.

c. TokenUsage / response-time records

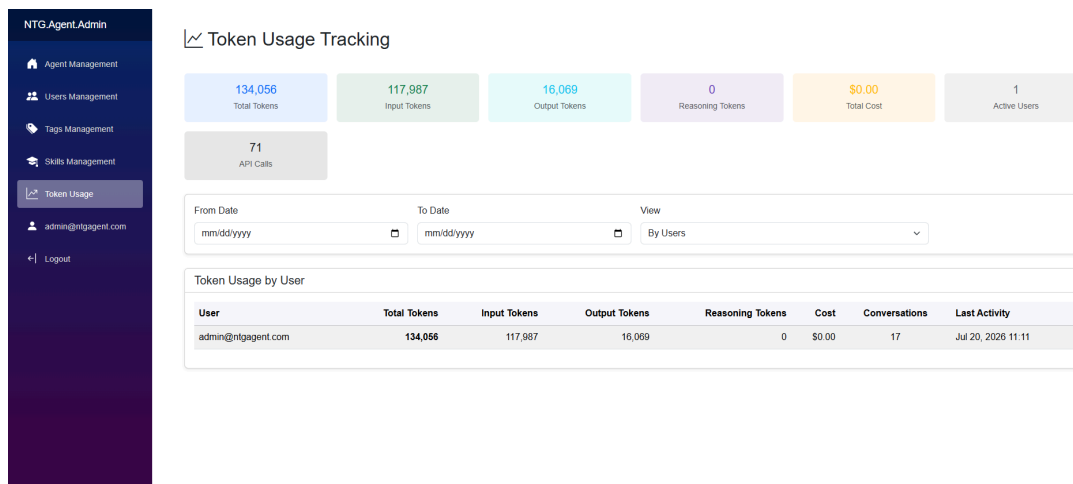


Figure 9: Track user token usage



Figure 10: Agent’s response-time and its element

d. Knowledge engine

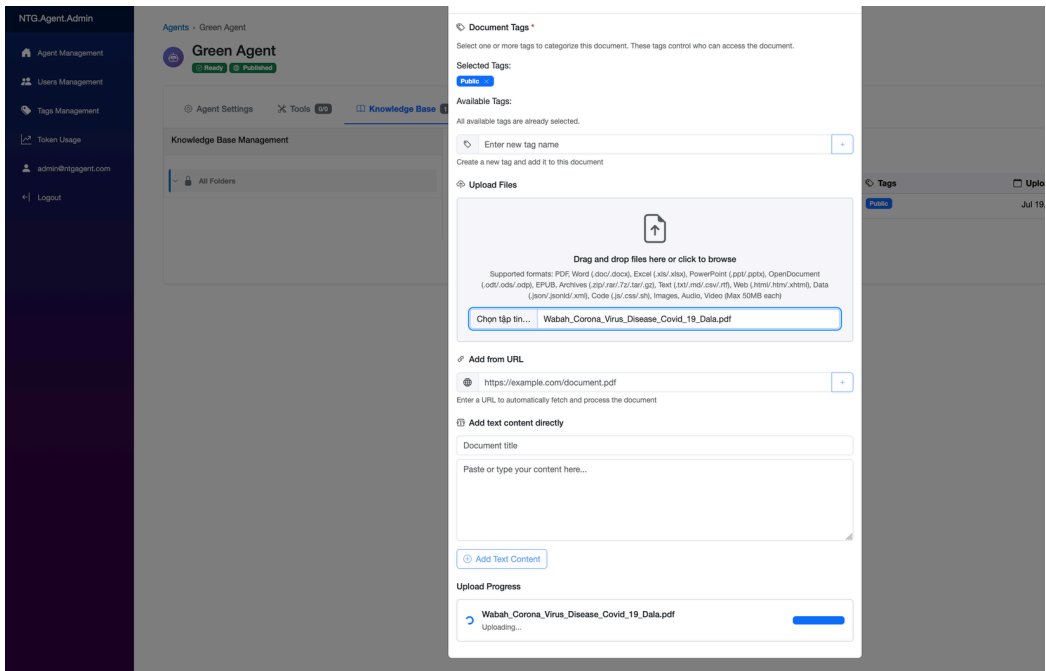


Figure 11: Upload a document for an agent

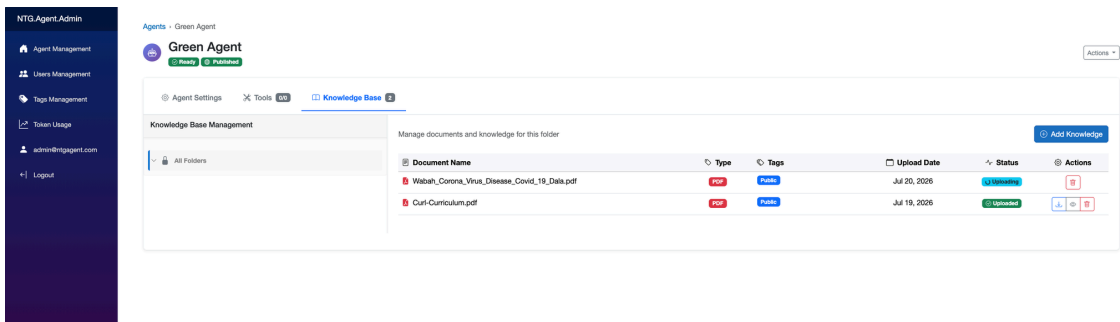


Figure 12: The UI indicates that the document is processed by LightRAG

e. Isolated stores. Two agents with different document libraries side by side

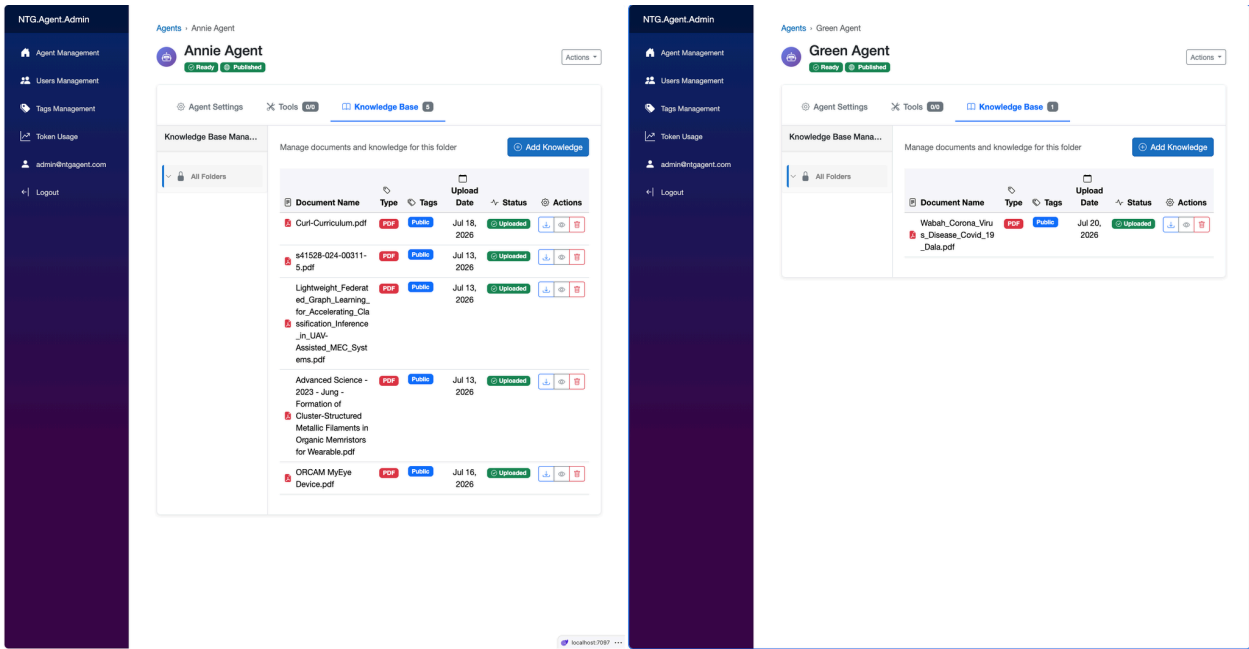


Figure 13: Isolate knowledge of the Annie Agent and the Green Agent

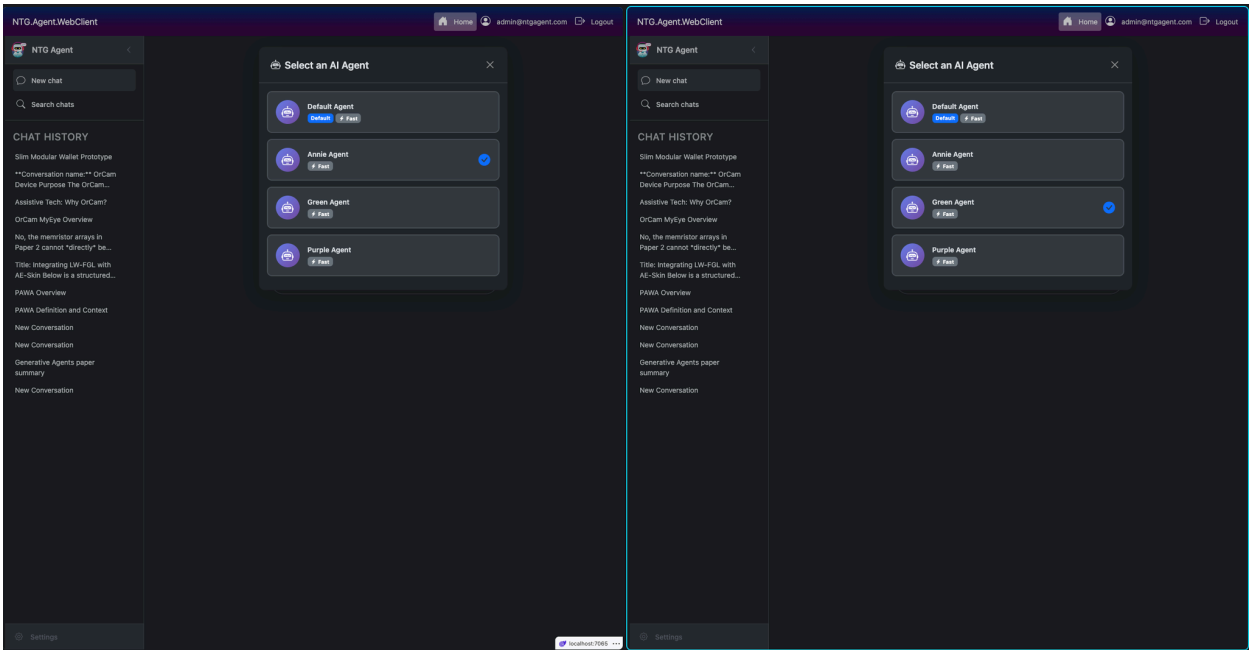


Figure 14: Ask Annie agent about an information that it does not have in its knowledge while the Green agent has it

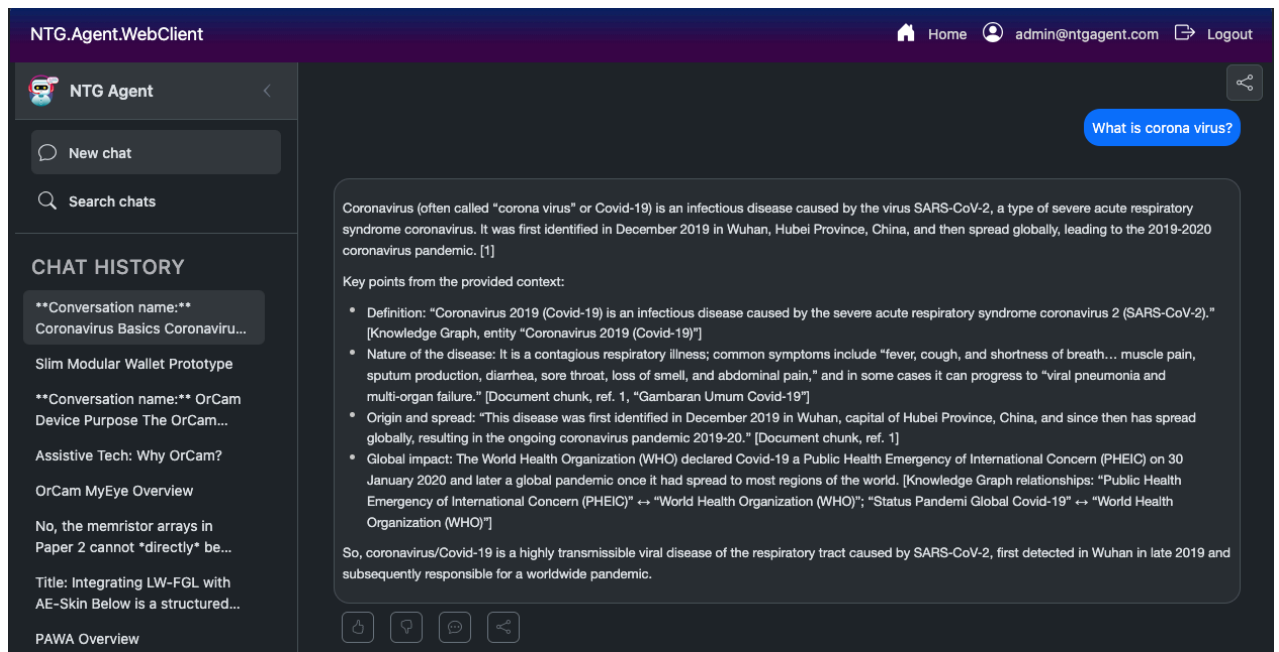


Figure 15: The Green agent can answer the question in detail thanks to the uploaded document

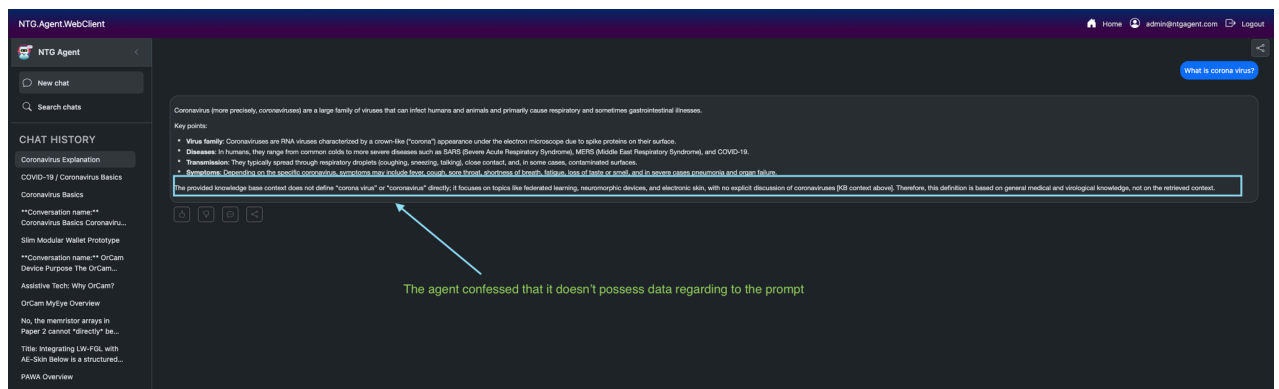


Figure 16: The Annie agent still answer to the prompt but with the confession that its knowledge does not have the data

f. AG-UI/A2UI

AG-UI feature:

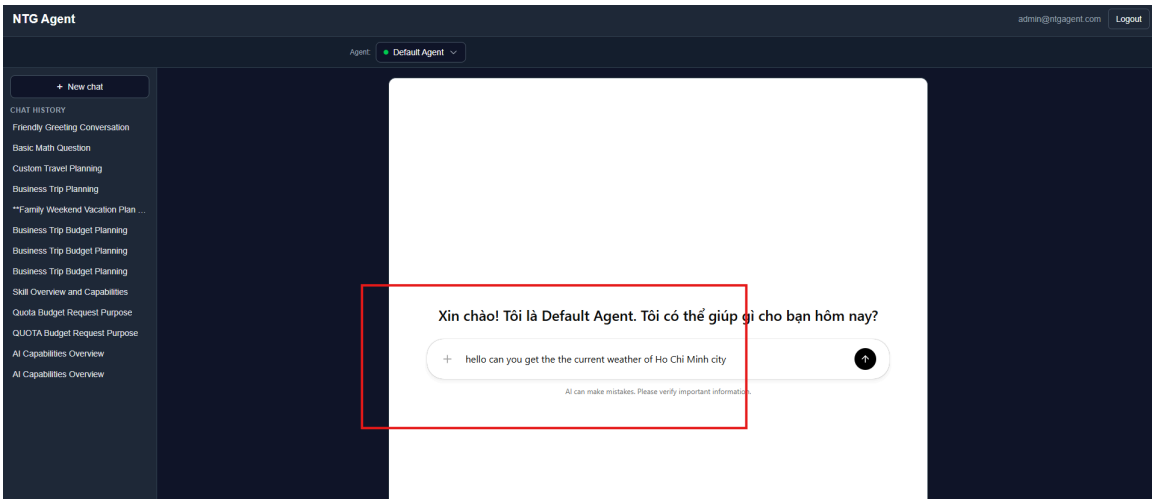


Figure 17: user prompt in the new webclient made specifically for AG-UI and A2UI

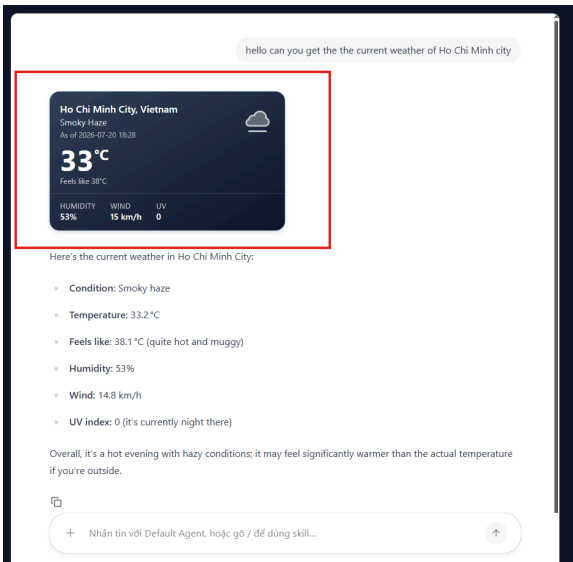


Figure 18: AG-UI feature puts information in a pre-design card.

A2UI Feature:

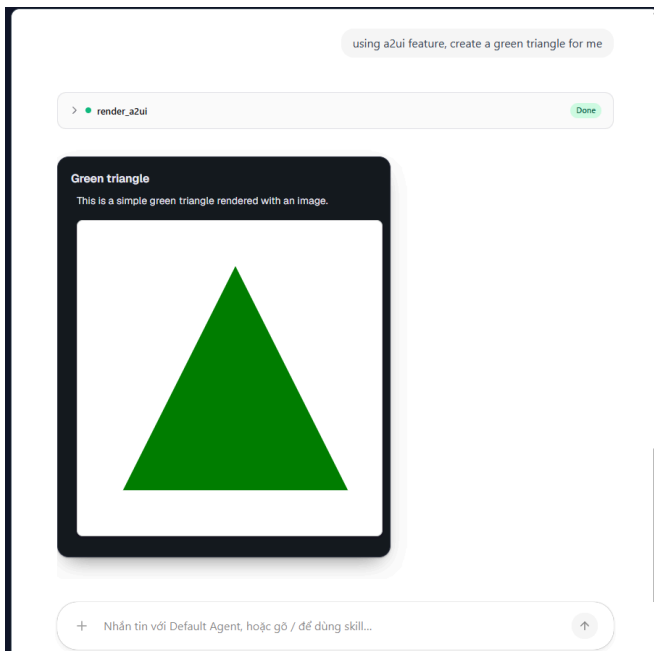


Figure 19: user asks agent to generate object from scratch and display it on chat box.

g. Engineering discipline

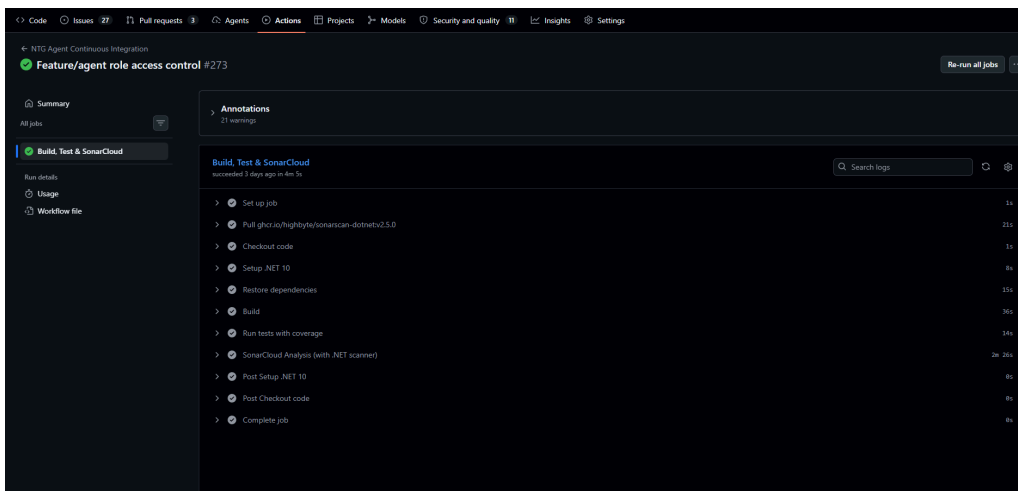


Figure 20: Every PR will get to check with SonarCloud quality gate green

B. GanttChart



Figure 21: Full project timeline (Phases 2-4) with milestones and dependencies